Deploying a Scalable Web Application on AWS using Auto Scaling & Load Balancer (Step-by-Step Guide)

Introduction

Modern web applications must be scalable, highly available, and fault-tolerant. This guide demonstrates how to design and deploy a simple web application on AWS using an Auto Scaling Group (ASG) behind an Elastic Load Balancer (ELB)

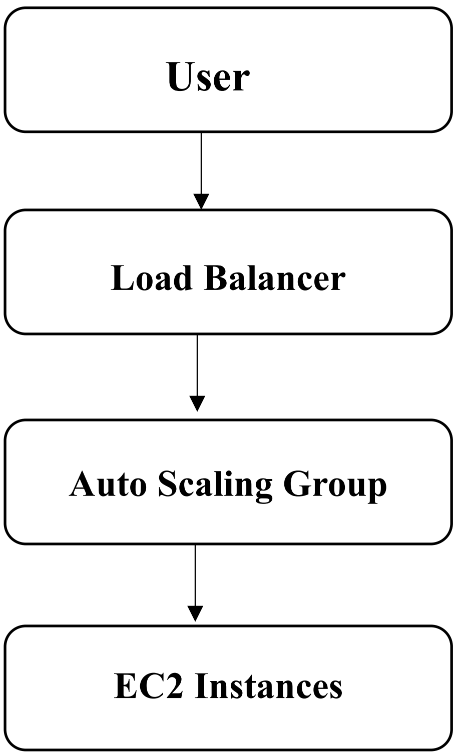
Key Concepts

- **Auto Scaling**
  Auto Scaling automatically adjusts the number of EC2 instances in response to demand, ensuring consistent performance at optimal cost.

- **Load Balancer**
  A Load Balancer distributes incoming application traffic across multiple targets (such as EC2 instances) in multiple Availability Zones.

In simple terms, Auto Scaling adds more servers when traffic increases and removes them when traffic decreases and Load Balancer acts like a traffic controller that ensures no single server is overloaded.

Request Flow

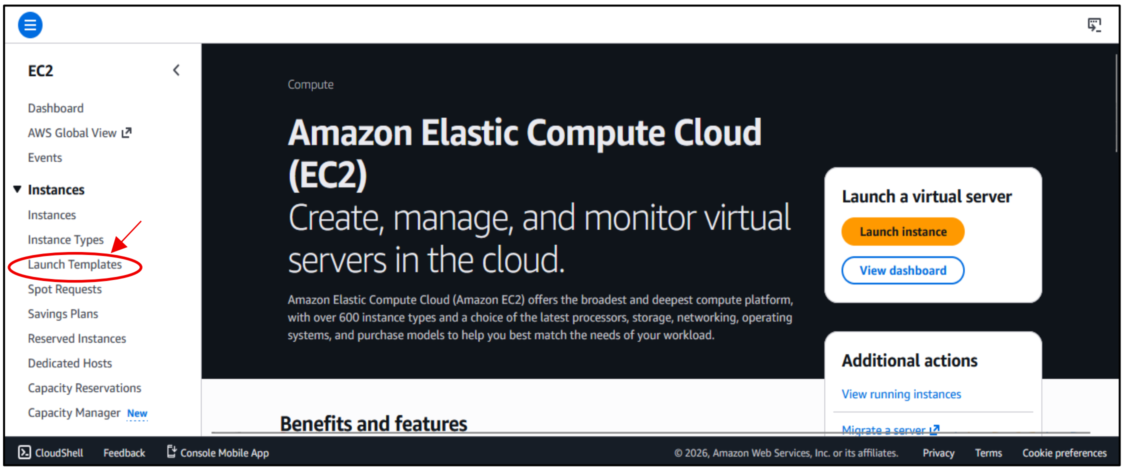


Step–by–Step Implementation

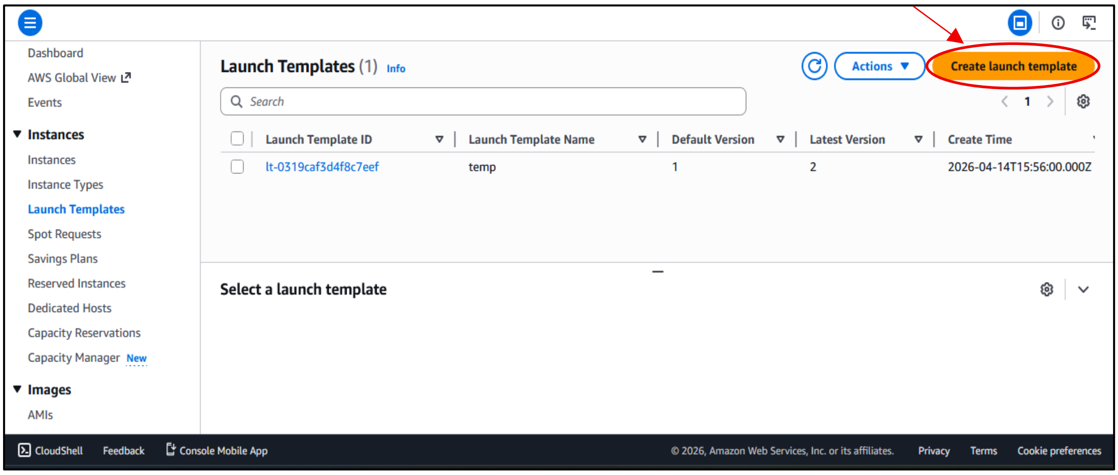
Step 1 : Create a Launch Template

A Launch Template defines how EC2 instances should be created automatically.

1. Go to **Amazon EC2 → Launch Templates**

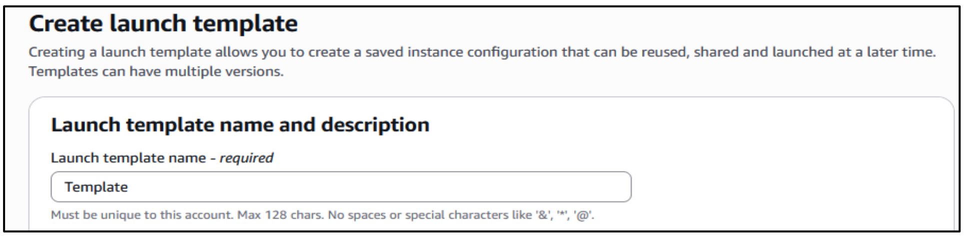


2. Click **Create Launch Template**

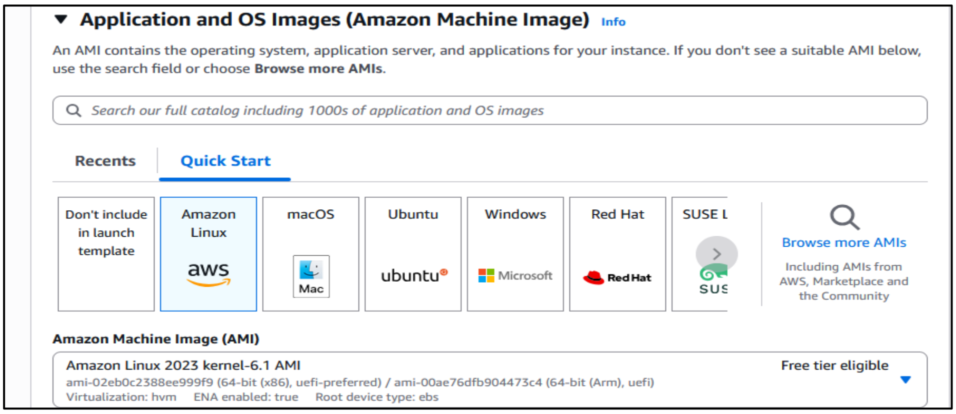


3. Configure Template

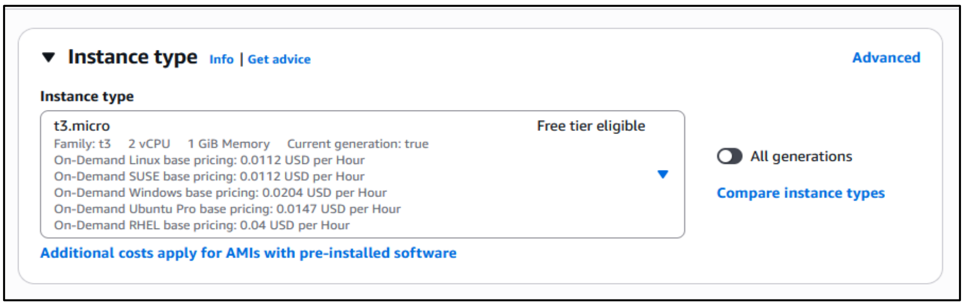
- **Template name**: (any name)



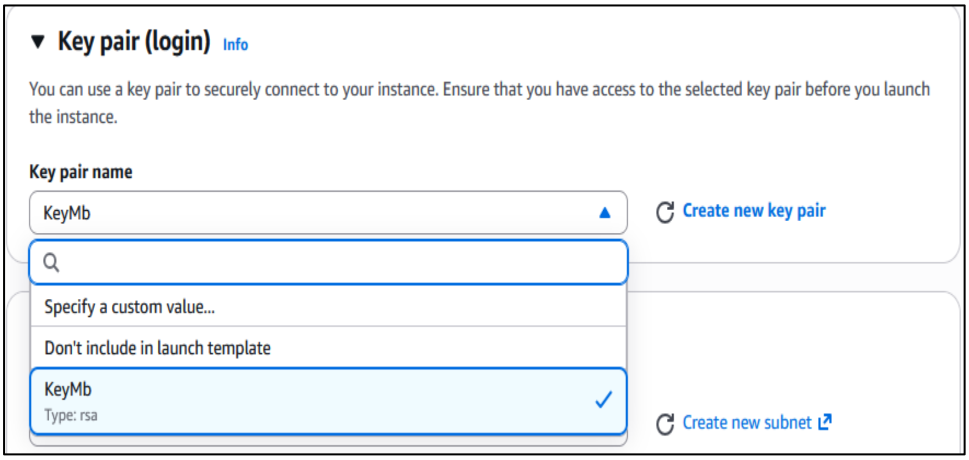
- **AMI**: Select your created AMI



- **Instance type**: t3.micro (or available alternative)



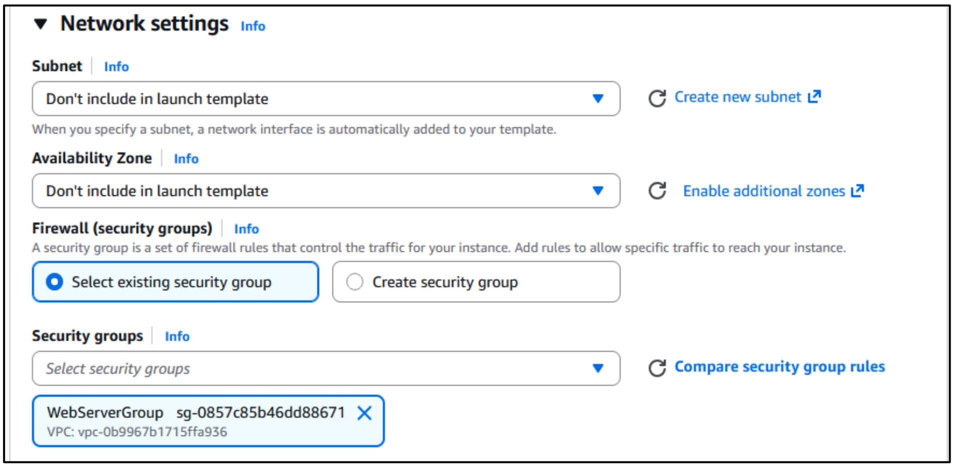
- **Key pair**: Select existing or create a new one



4. Network Settings

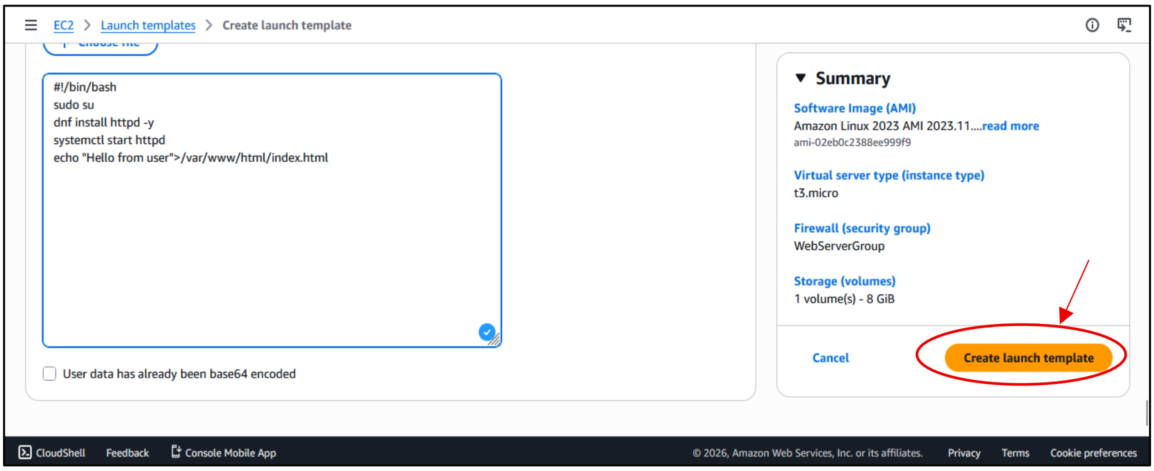
   Security Group:

   - Allow HTTP (Port 80)
   - Allow SSH (Port 22)

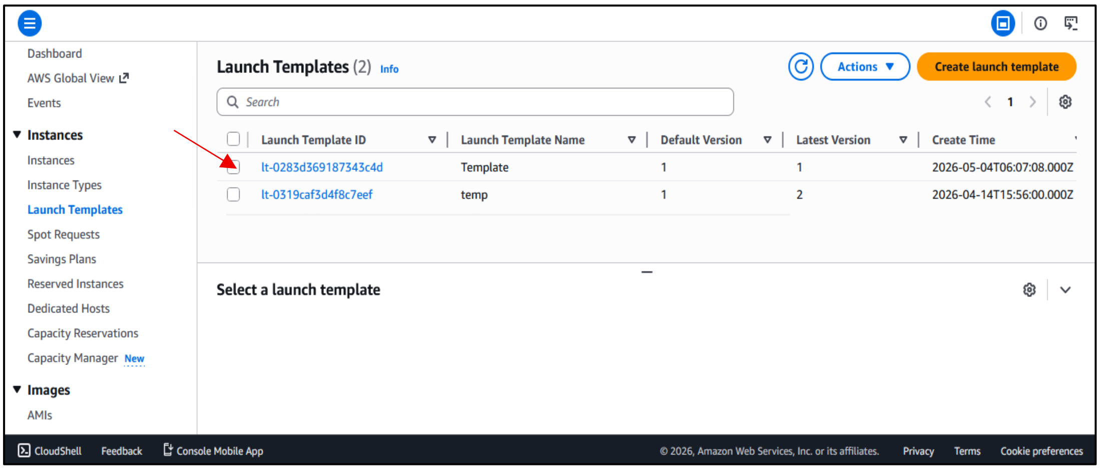


5. Add **User Data Script** in **Advanced Details**

6. Click **Create Launch Template**



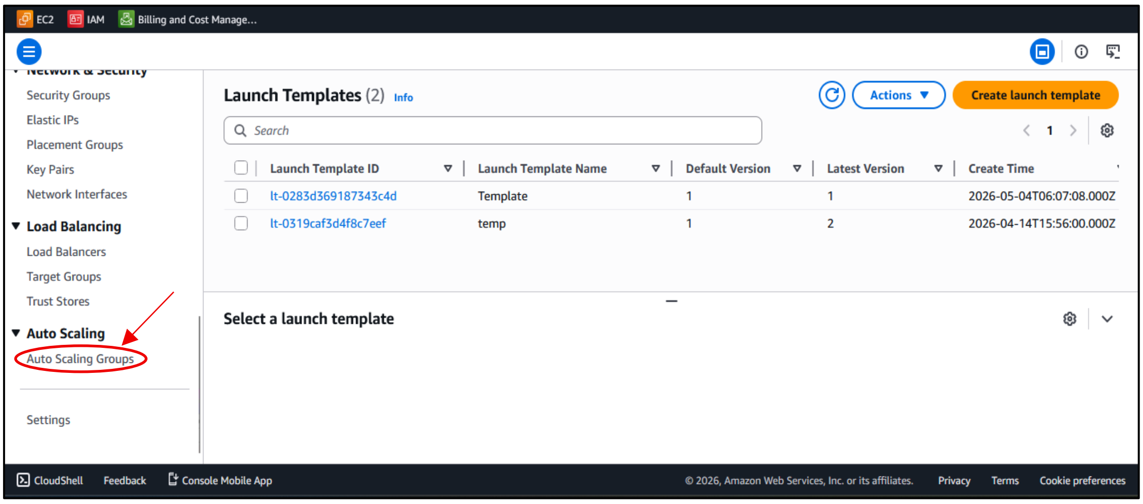
Template has been launched successfully.



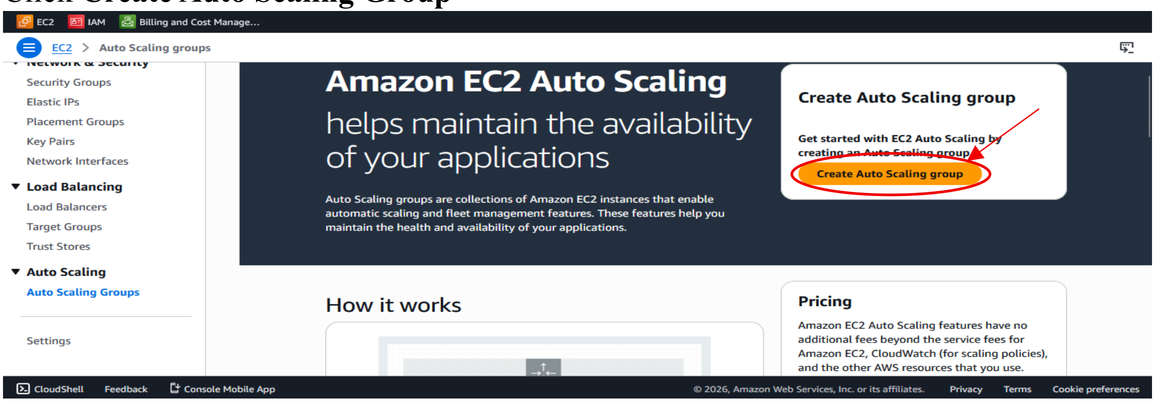
Step 2 : Create an Auto Scaling Group

An Auto Scaling Group automatically launch instances, maintains desired capacity and distributes traffic via Load Balancer.

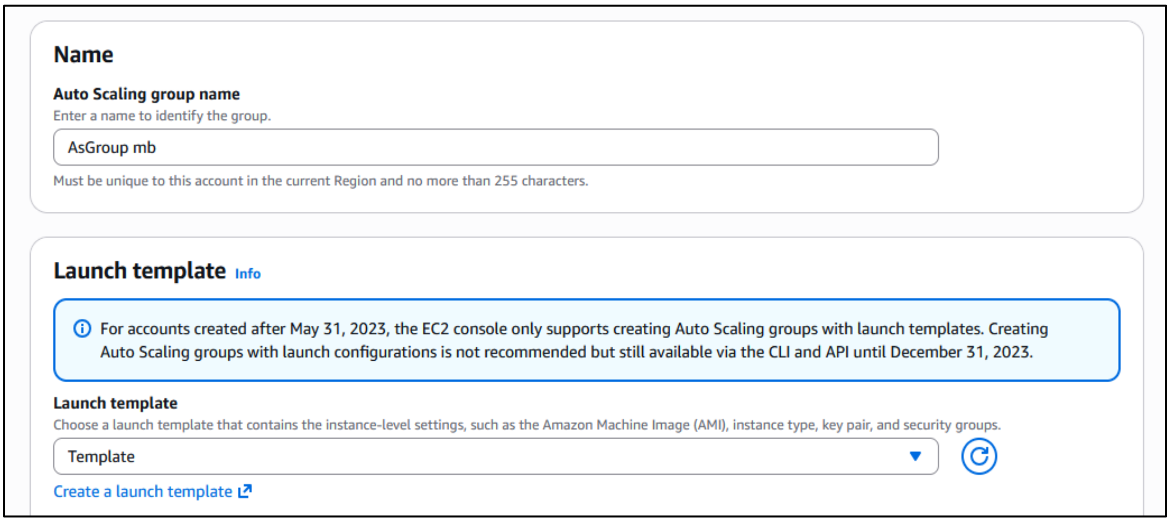
1. Go to **Auto Scaling Groups**



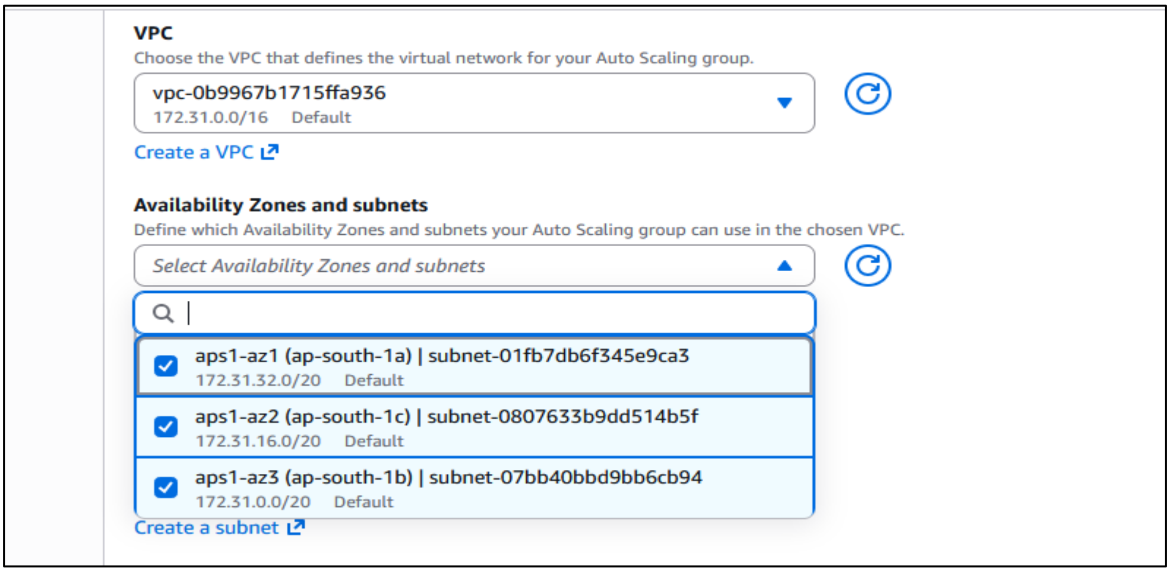
2. Click **Create Auto Scaling Group**

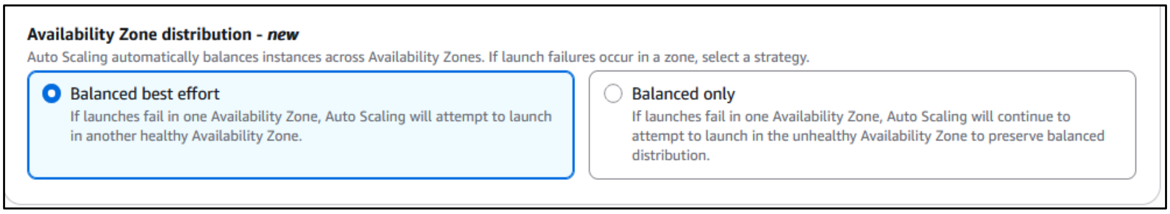


3. Select your **Launch Template**

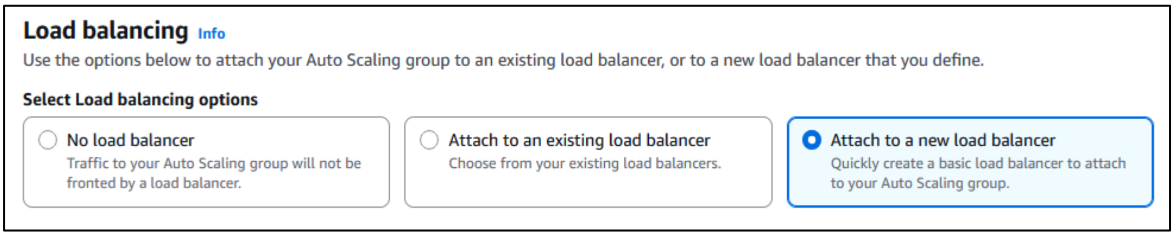


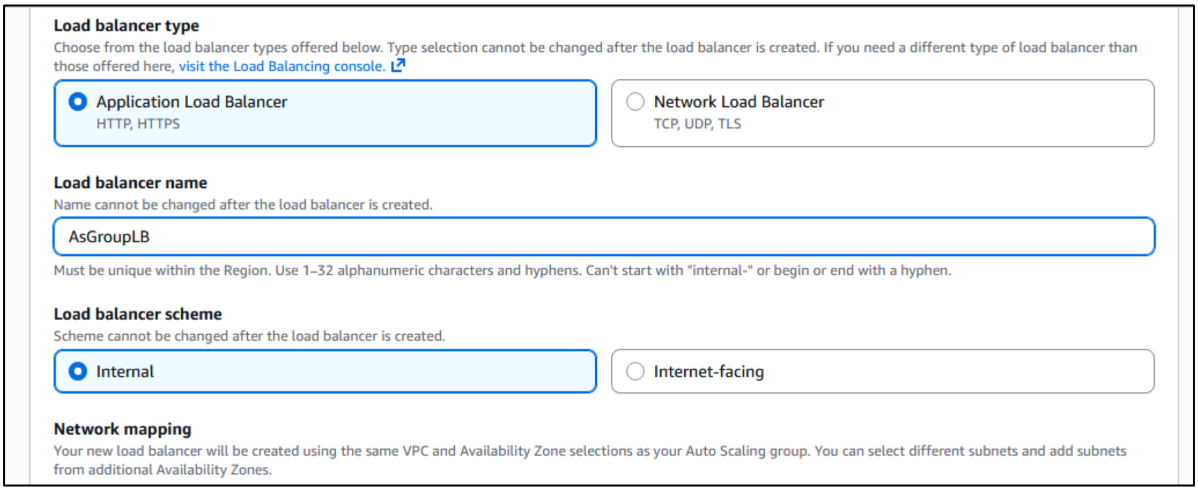
4. Configure **Network Settings**



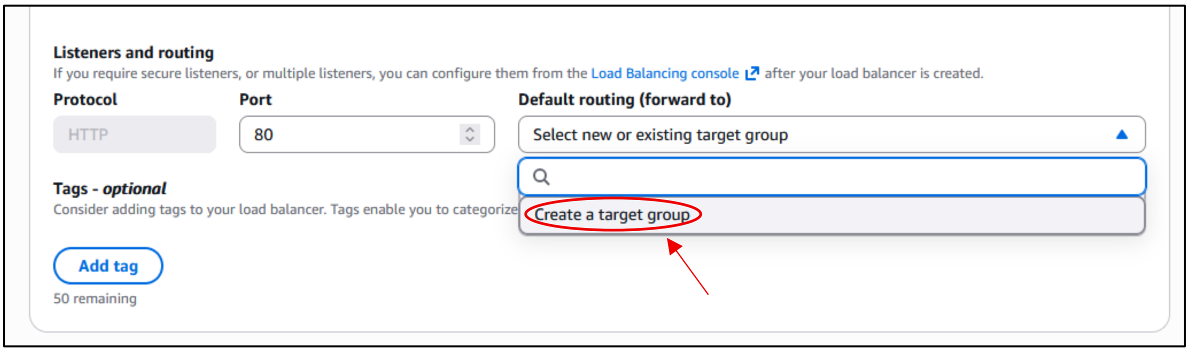


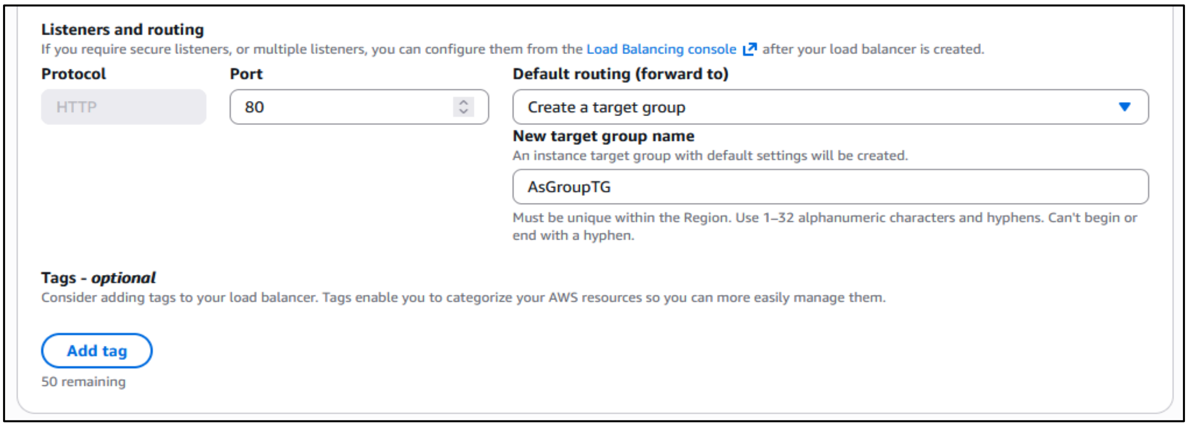
5. Configure **Load Balancer**



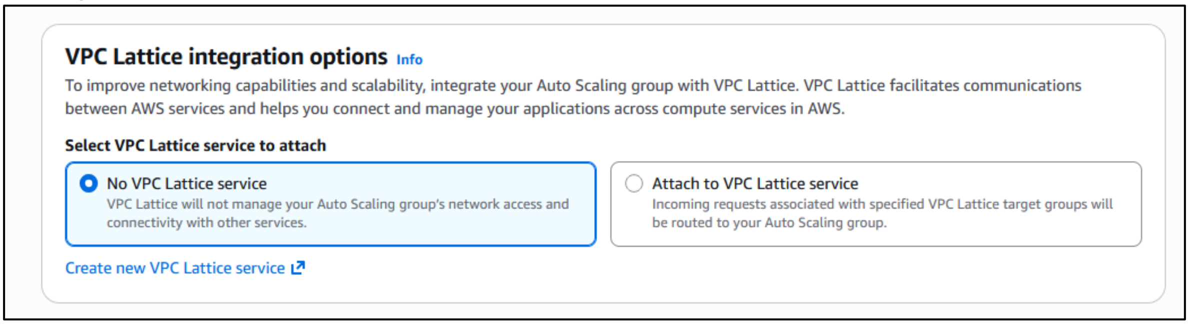


6. Configure **Target Group**



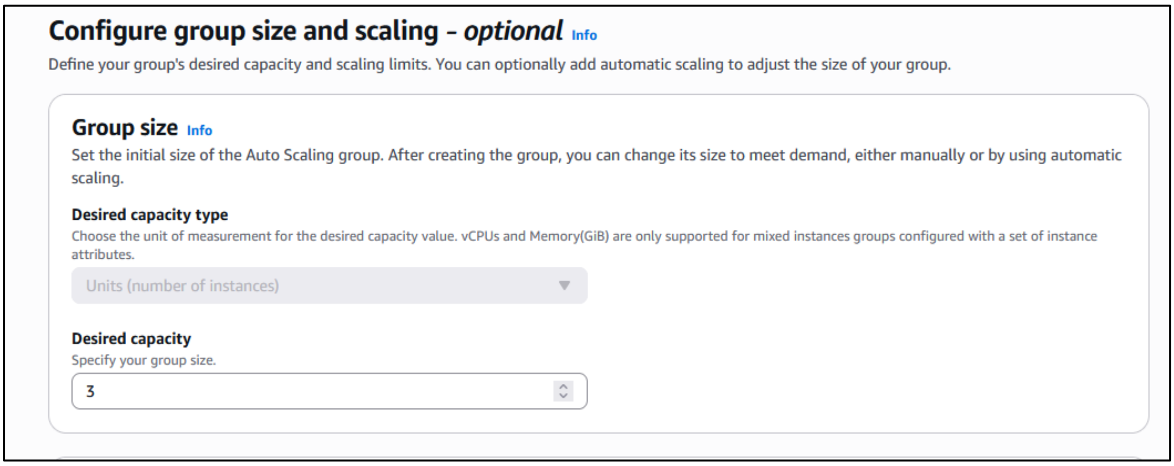


7. Configure **VPC Lattice**

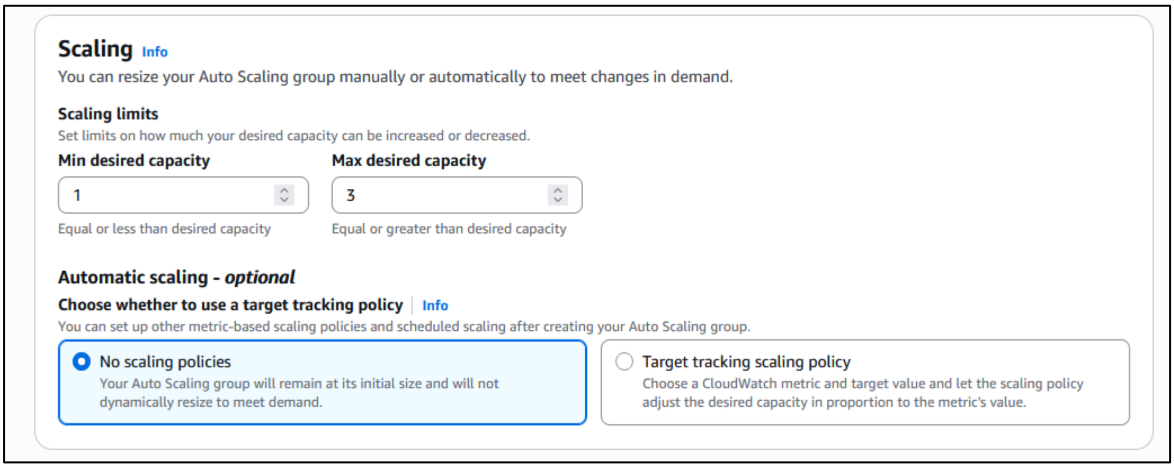


8. Configure **Group Size** and **Scaling**

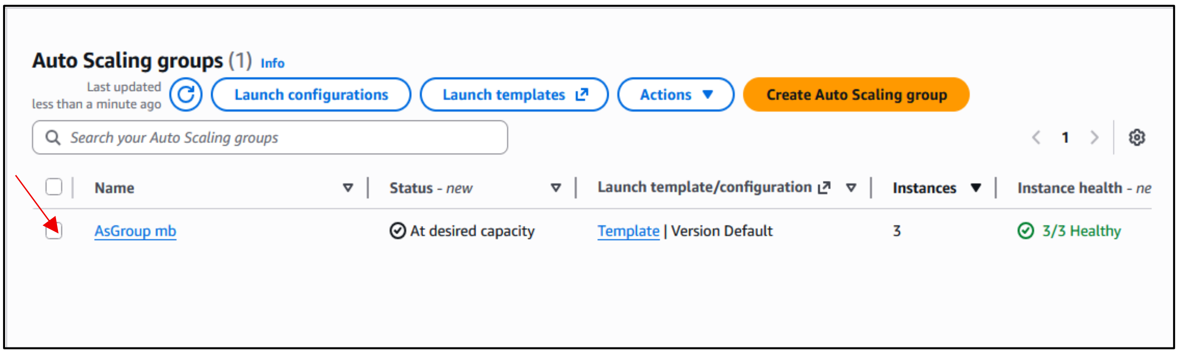
- Set **Desired capacity**



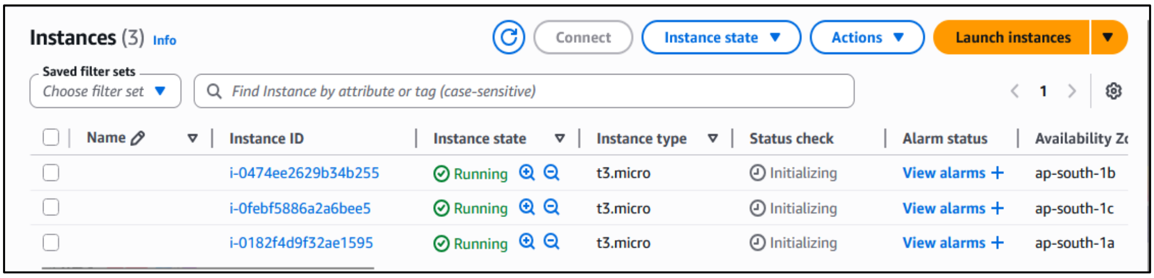
- Set **Min Desired Capacity** and **Max Desired Capacity**

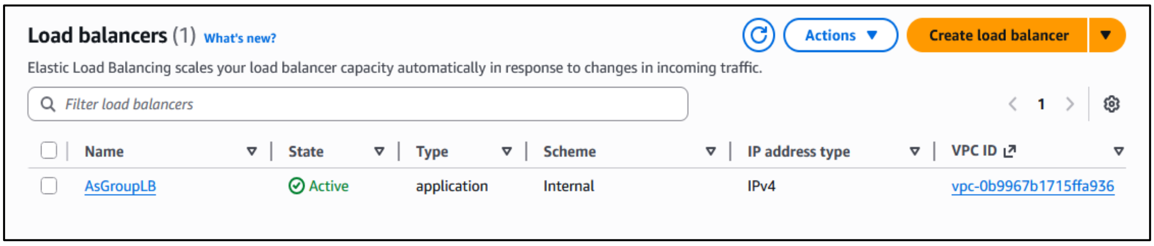


9. Create **Auto Scaling Group**



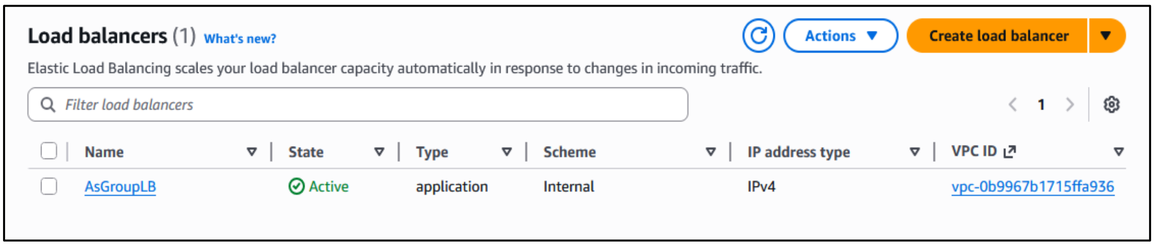
10. Verifying **Instances, Load Balancer and Target Group**



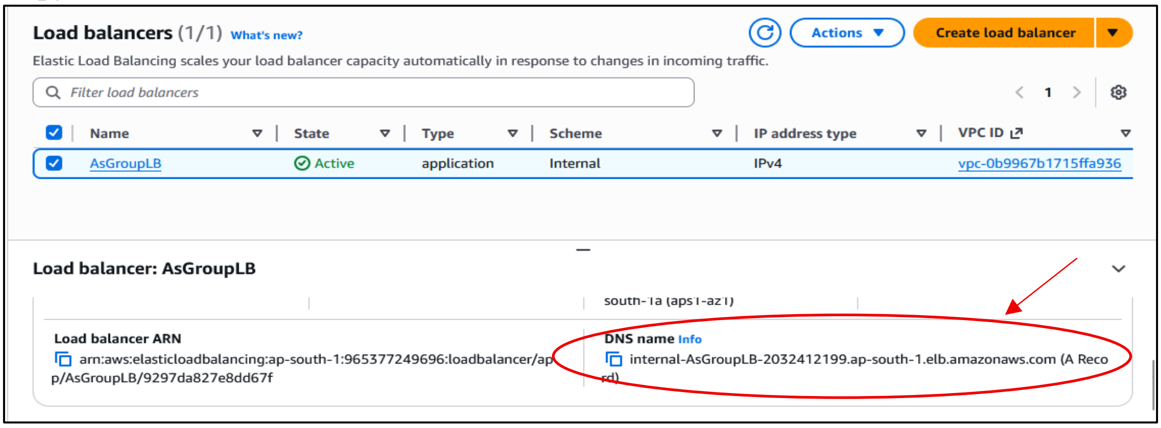


Step 4 : Verify Deployment

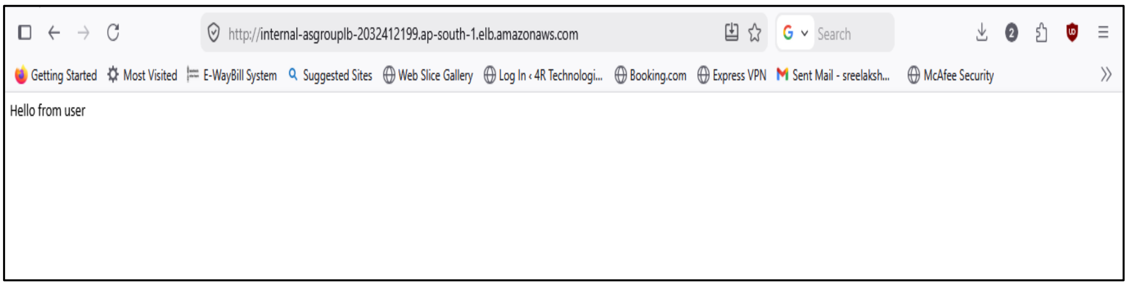
1. Navigate to EC2 → Load Balancers



2. Copy the DNS name



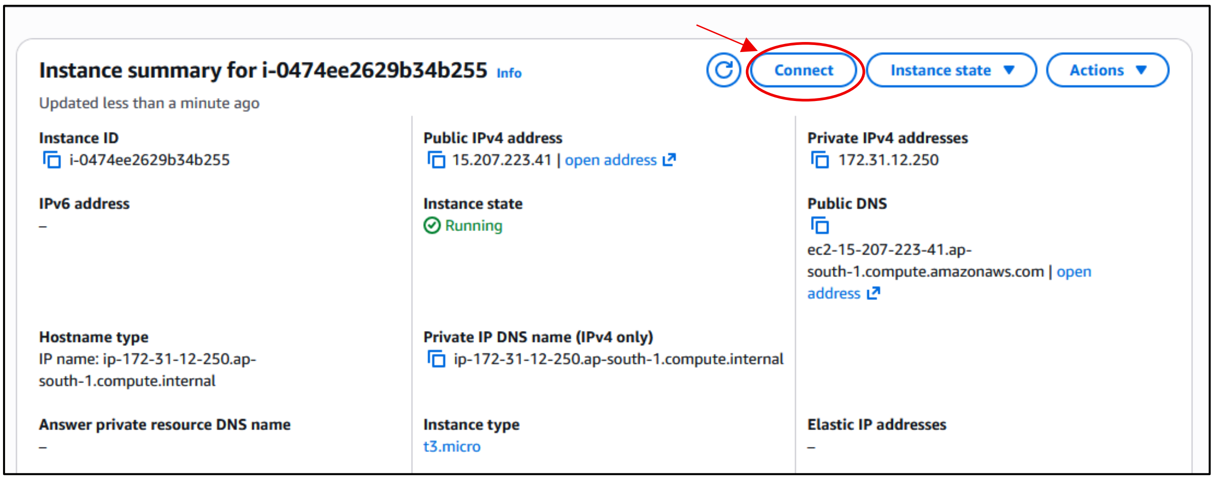
3. Open in browser



Step 5 : Validate Auto Scaling

1. Navigate to **EC2 Instances**

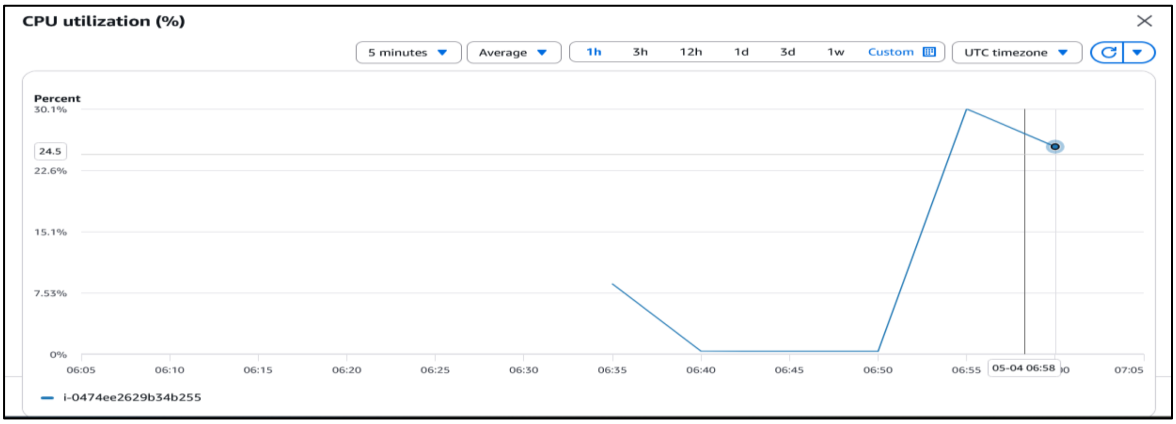
2. Click on any instance and connect



3. Connect to an instance and run:

```
yum install stress -y
stress -c 10 -t 300
```

4. Click **Monitoring** in the instance



Conclusion

This project demonstrated how to deploy a scalable web application using a Launch Template, Auto Scaling Group, and Application Load Balancer on AWS. The system automatically adjusts the number of instances based on CPU utilization and distributes traffic efficiently, ensuring high availability.

Overall, this setup provides a simple yet effective example of how cloud applications can handle varying workloads with minimal manual intervention.